

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA CƠ KHÍ - BỘ MÔN CƠ ĐIỆN TỬ



NHẬP MÔN THỊ GIÁC MÁY TÍNH
BÀI TẬP LỚN

LỚP TN01 - HK 251

GVHD: PGS.TS. LÊ ĐỨC HẠNH

SINH VIÊN THỰC HIỆN:

STT	Họ và tên	MSSV
1	Võ Hữu Dư	2210604
2	Đào Trọng Chân	2210350
3	Dương Quang Duy	2210497
4	Trần Quang Đạo	2210647

TP.HCM, Ngày 23 tháng 11 năm 2025

Mục lục

1	Câu 3	2
1.1	Calib Camera	2
1.1.1	Mục tiêu	2
1.1.2	Cơ sở lý thuyết	2
1.1.3	Các bước thực hiện	3
1.2	Tính Độ Dài Vật	6
1.2.1	Mục tiêu bài tập	6
1.2.2	Cơ sở lý thuyết	6
1.2.3	Các bước thực hiện	6
1.2.4	Kết quả thực nghiệm	8
2	Câu 5	11
2.1	Cơ sở lý thuyết	11
2.2	Các bước thực hiện	12

Danh sách bảng

1	Bảng dữ liệu tọa độ các điểm dùng để calib	15
---	--	----

Danh sách hình vẽ

1	Ảnh bàn cờ gốc dùng cho hiệu chỉnh camera.	3
2	Ảnh bàn cờ sau khi phát hiện và nối các góc nội bằng OpenCV.	4
3	Kết quả in ra màn hình: sai số RMS và ma trận nội tại K.	5
4	Kết quả các hệ số méo (distortion coefficients).	5
5	Tham số ngoại tại cho ảnh bàn cờ 1.	5
6	Thẻ quảng cáo và thước đo – bố trí theo chiều dài.	7
7	Thẻ và thước đo – bố trí theo chiều rộng.	7
8	Hình ảnh phát hiện 4 góc của hình.	8
9	Hình ảnh tọa độ pixel 4 góc.	9
10	Hình ảnh quy đổi tọa độ thực 4 góc.	9
11	Độ dài các cạnh.	9
12	Kích thước cuối cùng của vật.	9
13	Mặt phẳng hệ tọa độ dùng để calib camera trực tiếp	12
14	Dùng thước dây đo độ cao từ camera đến mặt phẳng hệ tọa độ	13
15	Thứ tự đánh dấu các điểm dùng để calib trên giao diện GUI	15
16	Kết quả ma trận calib thu được từ phương pháp trực tiếp	17
17	Các điểm dùng để kiểm tra lại, độ cao $h = 492\text{mm}$	18
18	Kết quả tọa độ các điểm trong không gian thực	19
19	Kết quả khoảng cách giữa hai điểm P1 và P2	19
20	Độ cao $h_1 = 467\text{ mm}$	20
21	Độ cao $h_2 = 475\text{ mm}$	20
22	Độ cao $h_3 = 492\text{ mm}$	20

1 Câu 3

Thực hiện calib camera sau đó so sánh tọa độ thật, độ dài của vật và tọa độ, độ dài của vật tính toán sau khi calib.

1.1 Calib Camera

1.1.1 Mục tiêu

Bài tập này nhằm hiệu chỉnh (calibrate) một camera sử dụng mẫu bàn cờ (checker-board). Thông qua quá trình hiệu chỉnh, ta xác định các tham số nội tại (intrinsic parameters), các hệ số méo ống kính (distortion coefficients) và tham số ngoại tại (extrinsic parameters) mô tả tư thế của bàn cờ trong hệ tọa độ camera. Các tham số này là đầu vào cho các bài toán như hiệu chỉnh méo (undistortion), đo kích thước vật thể, tái tạo 3D, mô phỏng stereo, v.v.

1.1.2 Cơ sở lý thuyết

- **Mô hình camera lỗ kim**

Mô hình camera đơn giản hóa như một lỗ kim (pinhole), mô tả mối quan hệ giữa không gian 3D và mặt phẳng ảnh 2D.

Công thức chiếu

$$\begin{bmatrix} s \cdot u \\ s \cdot v \\ s \end{bmatrix} = K_{int} * M_{ext} \begin{bmatrix} X_w \\ Y_w \\ Z_w \\ 1 \end{bmatrix} \quad (1)$$

Trong đó

- (X_w, Y_w, Z_w) : Tọa độ điểm trong không gian.
- (u, v) : Tọa độ điểm ảnh.
- $K_{int} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$: Ma trận nội tại chứa thông tin tiêu cự, tâm ảnh, hệ số lệch trục.
- $M_{ext} = \begin{bmatrix} R & t \\ 0_{1 \times 3} & 1 \end{bmatrix}$: Ma trận thông số ngoại dùng để chuyển tọa độ vật từ tọa độ toàn cục về tọa độ camera.

- **Méo hình của ống kính**

- Ống kính thực gây ra méo hình so với mô hình lý tưởng. Có hai loại méo chính:
 - + **Méo hướng tâm (radial)**: làm các đường thẳng gần rìa bị cong, được mô tả bởi các hệ số k_1, k_2, k_3 .
 - + **Méo tiếp tuyến (tangential)**: do ống kính lắp lệch tâm hoặc cảm biến bị nghiêng, mô tả bởi p_1, p_2 .
- OpenCV gom các hệ số này trong vector:

$$dist = [k_1, k_2, p_1, p_2, k_3, \dots] \quad (2)$$

- **Bài toán hiệu chỉnh camera**

Cho nhiều ảnh bàn cờ, mỗi ảnh ta có:

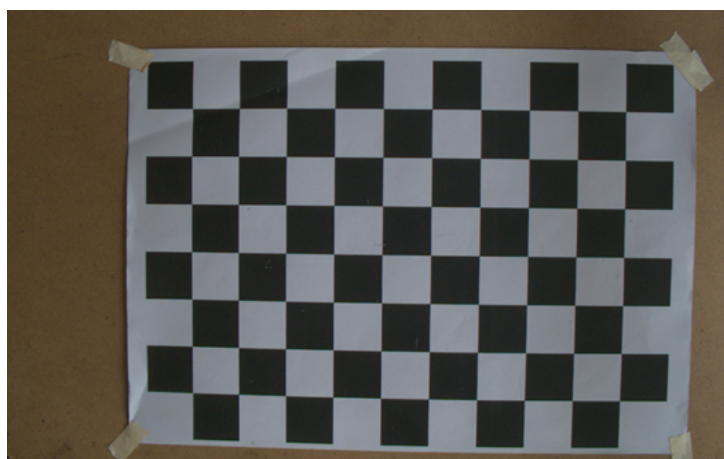
- **Object points:** tọa độ 3D các góc nội của bàn cờ, biết trước trong đơn vị mm.
- **Image points:** tọa độ 2D (u, v) của các góc nội tương ứng trên ảnh.

Hàm `cv2.calibrateCamera` sẽ tìm $K, dist$, cùng với R_i, t_i cho từng ảnh sao cho lỗi chiếu lại (reprojection error) giữa các điểm ảnh quan sát và các điểm ảnh dự đoán là nhỏ nhất. Giá trị RMS càng nhỏ thì kết quả hiệu chỉnh càng chính xác.

1.1.3 Các bước thực hiện

Camera: camera số chụp ảnh tĩnh. Mẫu bàn cờ được in trên giấy, dán lên mặt phẳng cứng như Hình 1. Sử dụng số góc nội `CHECKERBOARD = (7, 10)`. Cạnh mỗi ô vuông có kích thước `square_size_mm = 25 mm`.

Bộ ảnh hiệu chỉnh gồm nhiều ảnh bàn cờ được chụp ở các tư thế và khoảng cách khác nhau, giúp dữ liệu phong phú và phủ đều cảm biến. Ở đây nhóm sử dụng 25 ảnh bàn cờ để thực hiện hiệu chỉnh.



Hình 1: Ảnh bàn cờ gốc dùng cho hiệu chỉnh camera.

- **Bước 1:** Xây dựng hệ tọa độ bàn cờ (object points).

Ta giả sử toàn bộ bàn cờ nằm trên mặt phẳng $Z = 0$. Góc tọa độ $(0, 0, 0)$ đặt tại một góc nội của bàn cờ. Trục X chạy theo chiều ngang bàn cờ, trục Y theo chiều dọc. Sử dụng `np.mgrid` sinh ra lưới các chỉ số $(0 \dots 6, 0 \dots 9)$, sau đó nhân với kích thước ô vuông 25 mm để thu được 70 điểm 3D.

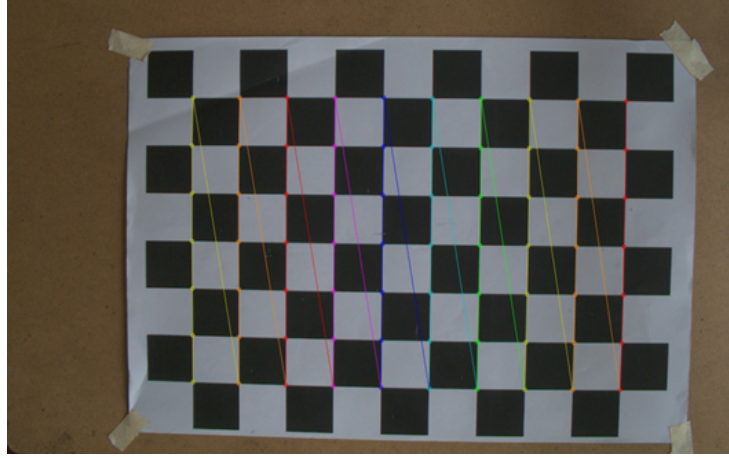
Lưới object points này là như nhau cho mọi ảnh, vì hình học bàn cờ không đổi. Mỗi ảnh sẽ dùng cùng một tập object points và một tập image points riêng.

- **Bước 2:** Tìm các góc bàn cờ trên ảnh (image points).

Các ảnh trong thư mục được lần lượt đọc và chuyển sang ảnh xám. Hàm `cv2.findChessboardCorners` với kích thước $(7, 10)$ được sử dụng để tìm vị trí các góc nội. Nếu nhận diện thành công (`ret = True`), vị trí góc được refine bằng `cv2.cornerSubPix` để đạt độ chính xác sub-pixel.

Kết quả refine được đưa vào `imgpoints`, trong khi object points tương ứng được

đưa vào `objpoints`. Đồng thời, chương trình vẽ các góc lên ảnh bằng `cv2.drawChessboardCorners` và lưu lại, giúp kiểm tra trực quan xem việc phát hiện góc có chính xác hay không.



Hình 2: Ảnh bàn cờ sau khi phát hiện và nối các góc nội bằng OpenCV.

- **Bước 3:** Gọi `cv2.calibrateCamera` và `cv2.getOptimalNewCameraMatrix` để tính các tham số nội tại, ngoại tại và hệ số méo.

Sau khi thu đủ dữ liệu, ta gọi `cv2.calibrateCamera(objpoints, imgpoints, (w, h), ...)` để tính các tham số nội tại và ngoại tại. Kết quả chính trong thí nghiệm là:

- Reprojection error (RMS) ≈ 0.499977 pixel (xấp xỉ 0.50 pixel).
- Ma trận nội tại K :

$$K = \begin{bmatrix} 2385.3882 & 0.0000 & 944.8986 \\ 0.0000 & 2387.7920 & 553.7859 \\ 0.0000 & 0.0000 & 1.0000 \end{bmatrix} \quad (3)$$

Suy ra:

$$f_x = 2385.3882 \text{ pixel}$$

$$f_y = 2387.7920 \text{ pixel}$$

$$c_x = 944.8986 \text{ pixel}$$

$$c_y = 553.7859 \text{ pixel}$$

f_x và f_y gần bằng nhau, cho thấy pixel gần vuông. Tâm ảnh (c_x, c_y) nằm gần trung tâm khung hình, phù hợp với cấu trúc cảm biến của camera.

Vector hệ số méo thu được là:

$$dist = [k_1, k_2, p_1, p_2, k_3] = [-0.202966, -0.000392, 0.000621, 0.001218, 0.975811] \quad (4)$$

Trong đó k_1, k_2 âm thể hiện méo dạng barrel nhẹ; k_3 dương mô tả méo bậc cao hơn; p_1, p_2 rất nhỏ nên méo tiếp tuyến không đáng kể.

```

Reprojection error (RMS) ~ 0.499977

1) Camera matrix (K):
[[2385.3882    0.    944.8986]
 [    0.    2387.792  553.7859]
 [    0.        0.        1.   ]]
-> fx = 2385.3882, fy = 2387.7920
-> cx = 944.8986, cy = 553.7859
  
```

Hình 3: Kết quả in ra màn hình: sai số RMS và ma trận nội tại K.

```

2) Distortion coefficients (k1, k2, p1, p2, k3, ...):
[-0.203 -0.0004 0.0006 0.0012 0.9758]
-> k1 = -0.202966, k2 = -0.000392, p1 = 0.000621, p2 = 0.001218, k3 = 0.975811
  
```

Hình 4: Kết quả các hệ số méo (distortion coefficients).

```

--- Ảnh #1 ---
Rotation vector (rvec): [0.0684 0.0147 1.5807]
Rotation matrix (R):
[[-0.0095 -0.9985 0.053 ]
 [ 0.9994 -0.0113 -0.0338]
 [ 0.0344 0.0526 0.998 ]]
Translation vector (tvec) [mm]: [134.5134 -55.6098 456.1358]
  
```

Hình 5: Tham số ngoại tại cho ảnh bàn cờ 1.

• Nhận xét:

Giá trị $RMS \approx 0.42$ pixel cho thấy kết quả hiệu chỉnh là tốt; sai số chiếu lại trung bình nhỏ hơn một pixel. Ma trận nội tại thu được hợp lý, tâm ảnh gần trung tâm khung hình. Các hệ số méo có trị số vừa phải, chứng tỏ ống kính có méo nhưng không quá nghiêm trọng và hoàn toàn có thể hiệu chỉnh bằng các hàm undistort của OpenCV. Một số nguồn sai số có thể tồn tại:

- Bàn cờ in trên giấy có thể bị cong hoặc nhăn.
- Một số ảnh có ánh sáng không đồng đều, làm giảm tương phản ô đen – trắng.
- Số lượng và độ đa dạng của ảnh có thể chưa tối ưu; nếu nhiều ảnh có góc chụp gần giống nhau thì thông tin hình học thu được không phong phú.

Để cải thiện, có thể tăng số ảnh, thay đổi nhiều hơn về góc nhìn và khoảng cách, sử dụng vật liệu in cứng và loại bỏ các ảnh bị mờ hoặc thiếu sáng.

1.2 Tính Độ Dài Vật

1.2.1 Mục tiêu bài tập

Trong bài thực hành này, mục tiêu là sử dụng kết quả hiệu chỉnh (calibration) của camera để đo kích thước thực của một vật thể phẳng đặt trên cùng mặt phẳng với bàn cờ hiệu chỉnh. Cụ thể, chương trình cần:

- Chuyển bốn góc từ tọa độ pixel sang tọa độ thế giới (mm) dựa trên các tham số nội/ngoại của camera.
- Tính chiều dài, chiều rộng thực tế của thẻ và so sánh với số đo bằng thước thực tế.

1.2.2 Cơ sở lý thuyết

- **Mô hình chiếu phối cảnh camera**

Quan hệ giữa điểm trong không gian thế giới $M_w = [X_w, Y_w, Z_w]^T$ và điểm trong hệ tọa độ camera $M_c = [X_c, Y_c, Z_c]^T$ được mô tả bởi:

$$M_c = R \cdot M_w + T \quad (5)$$

Trong đó R (3×3) là ma trận quay và T (3×1) là vector tịnh tiến từ hệ thế giới sang hệ camera (được lấy từ file hiệu chỉnh).

Từ hệ camera sang điểm ảnh (pixel), ta có quan hệ chiếu phối cảnh: $s \cdot m_p = K \cdot M_c$, với $m_p = [u, v, 1]^T$ là tọa độ pixel ở dạng đồng nhất, K là ma trận nội chứa các tham số tiêu cự và tâm ảnh.

- **Chuyển tọa độ pixel sang tọa độ thế giới trên mặt phẳng $Z_w = 0$**

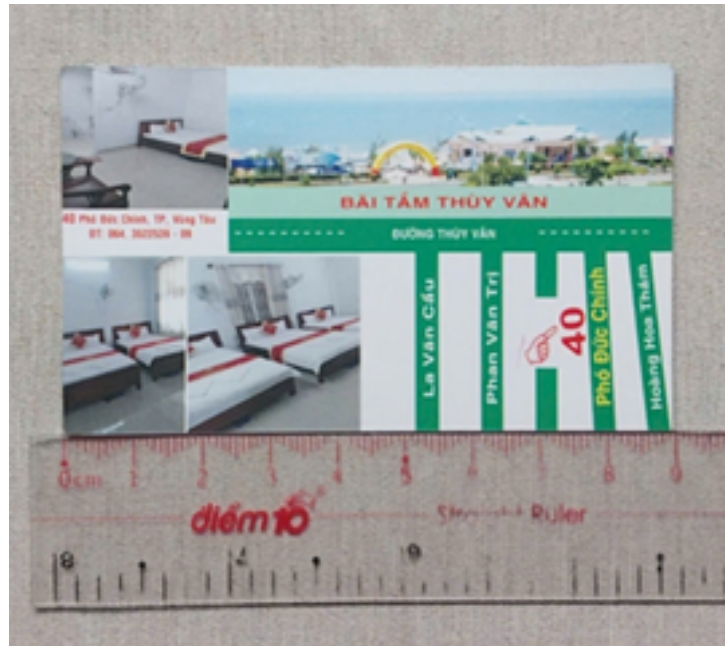
Nếu giả sử vật thể nằm trên mặt phẳng $Z_w = 0$ (mặt phẳng bàn cờ), ta có thể tính ngược tọa độ thế giới từ pixel. Từ $s \cdot m_p = K \cdot (R \cdot M_w + T)$, kết hợp với điều kiện $Z_w = 0$, ta suy ra:

$$M_w = R^{-1}(K^{-1} \cdot z_{cam} \cdot m_p - T) \quad (6)$$

Trong chương trình, z_{cam} (độ sâu trong hệ camera) được tính bằng cách xét thành phần thứ ba (trục Z_w) của M_w và ép điều kiện $Z_w = 0$ để giải ra z_{cam} . Sau đó, M_w được tính lại bằng công thức trên, cho ta tọa độ (X_w, Y_w, Z_w) tính theo mm.

1.2.3 Các bước thực hiện

- **Camera:** camera đã được hiệu chỉnh trước đó bằng phương pháp Zhang với bàn cờ chuẩn 7×10 góc nội.
- **File tham số hiệu chỉnh:** `calib_params.json`, lưu ma trận nội K và các cặp (`rvec`, `tvec`) cho nhiều tư thế chụp bàn cờ.
- **Vật đo:** thẻ quảng cáo giấy, kích thước thực tế xấp xỉ $90 \text{ mm} \times 54 \text{ mm}$ (quan sát bằng thước).



Hình 6: Thẻ quảng cáo và thước đo – bố trí theo chiều dài.



Hình 7: Thẻ và thước đo – bố trí theo chiều rộng.

- **Bước 1. Đọc tham số hiệu chỉnh từ file JSON**

Sau khi calib các thông số được lưu vào file JSON. Hàm `load_calib()` đọc ma

trận nội K , danh sách $rvecs$ và $tvecs$ từ `calib_params.json`. Vector quay $rvec$ được chuyển sang ma trận quay R bằng hàm `cv2.Rodrigues()`. Từ đó, ta thu được bộ tham số (K, R, T) ứng với một tư thế camera (`POSE_INDEX`).

- **Bước 2. Tiền xử lý ảnh và nhị phân hoá**

Ảnh màu được chuyển sang mức xám, làm mượt bằng `GaussianBlur` để giảm nhiễu. Sau đó, dùng phân ngưỡng Otsu (`cv2.THRESH_OTSU`) để tách nền – vật thể, thu được ảnh nhị phân. Các phép hình thái học mở (`opening`) và đóng (`closing`) với kernel 3×3 được áp dụng để loại nhiễu nhỏ và làm đầy các lỗ hổng trong vùng thể.

- **Bước 3. Tìm contour lớn nhất và hình chữ nhật bao nhỏ nhất**

Dùng `cv2.findContours()` với chế độ `RETR_EXTERNAL` để lấy các contour ngoài cùng. Contour có diện tích lớn nhất được giả định là thể cần đo. Hàm `cv2.minAreaRect()` được dùng để tìm hình chữ nhật bao nhỏ nhất (min-area rectangle) cho contour này, trả về toạ độ tâm, kích thước (w, h) và góc xoay. Từ đó, `cv2.boxPoints()` cho ta bốn đỉnh của hình chữ nhật này trong hệ pixel.

- **Bước 4. Chuyển bốn góc từ pixel sang toạ độ thế giới**

Bốn điểm góc (u, v) thu được từ `boxPoints()` được truyền vào hàm `pixel_to_world_on_plane()`. Hàm này triển khai công thức nghịch $M_w = R^{-1}(K^{-1} \cdot z_{cam} \cdot m_p - T)$, trong đó z_{cam} được giải bằng cách áp điều kiện $Z_w = 0$. Kết quả là toạ độ (X_w, Y_w, Z_w) của từng góc trong hệ thế giới, đơn vị mm.

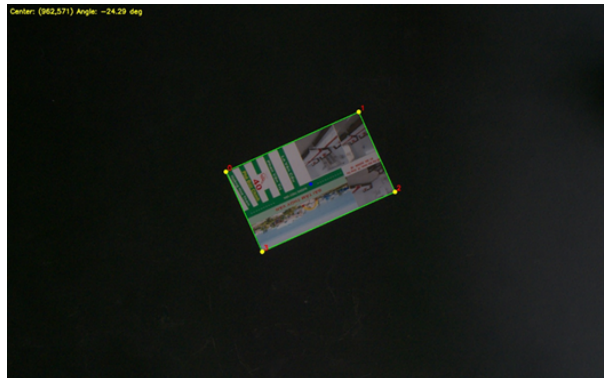
- **Bước 5. Tính chiều dài và chiều rộng thể**

Sau khi có bốn điểm trong không gian thế giới, khoảng cách giữa các cặp đỉnh liên tiếp được tính bằng chuẩn Euclid. Trong bốn cạnh, cạnh dài nhất được coi là chiều dài, cạnh ngắn nhất là chiều rộng của thể. Các giá trị này là kết quả đo kích thước thực tế dựa trên hiệu chỉnh camera.

1.2.4 Kết quả thực nghiệm

- **Toạ độ bốn góc trong hệ pixel**

Bốn góc của thể trong ảnh (theo thứ tự `boxPoints`) được hiển thị trong Hình 9.



Hình 8: Hình ảnh phát hiện 4 góc của hình.

```
Pixel corners (u, v) của hình:
Corner 0: u = 693, v = 532
Corner 1: u = 1114, v = 342
Corner 2: u = 1229, v = 596
Corner 3: u = 808, v = 786
```

Hình 9: Hình ảnh tọa độ pixel 4 góc.

- **Toạ độ bốn góc trong hệ thế giới (mm)**

Sau khi chuyển đổi sang hệ tọa độ thế giới trên mặt phẳng $Z_w = 0$, ta thu được kết quả trong Hình 10.

```
World coordinates (Xw, Yw, Zw) quy đổi từ 4 góc (mm):
Corner 0: Xw = 53.461, Yw = 183.652, Zw = -0.000
Corner 1: Xw = 15.795, Yw = 101.756, Zw = 0.000
Corner 2: Xw = 64.724, Yw = 78.924, Zw = -0.000
Corner 3: Xw = 103.024, Yw = 160.629, Zw = -0.000
```

Hình 10: Hình ảnh quy đổi tọa độ thực 4 góc.

- **Kích thước ước lượng của thẻ tính theo Euclid**

Khoảng cách giữa các cặp góc liên tiếp được tính bằng công thức Euclid:

```
Edge lengths (mm) theo thứ tự box:
Edge 0-1: 90.143 mm
Edge 1-2: 53.994 mm
Edge 2-3: 90.236 mm
Edge 3-0: 54.649 mm
```

Hình 11: Độ dài các cạnh.

```
>>> ƯỚC LƯỢNG KÍCH THƯỚC CARD (theo mặt phẳng bàn cờ):
Chiều dài (max cạnh): 90.236 mm
Chiều rộng (min cạnh): 53.994 mm
```

Hình 12: Kích thước cuối cùng của vật.

- **So sánh với số đo thực tế**

Dựa trên hình ảnh thước đo kèm theo (Hình 6 và Hình 7), kích thước thực của thẻ xấp xỉ: chiều dài 90 mm và chiều rộng 54 mm. So sánh với kết quả đo bằng camera:

- **Chiều dài:** 90.236 mm so với 90 mm $\rightarrow$ sai số khoảng 0.236 mm ($\sim 0.26\%$).
- **Chiều rộng:** 53.994 mm so với 54 mm $\rightarrow$ sai số khoảng 0.006 mm ($\sim 0.01\%$).

Sai số rất nhỏ, cho thấy quá trình hiệu chỉnh và đo đạc hoạt động tốt.

2 Câu 5

Thực hiện calib camera bằng phương pháp trực tiếp sau đó tìm lại ma trận thông số nội và ngoại, so sánh với phương pháp gián tiếp đã làm ở câu 3.

2.1 Cơ sở lý thuyết

Quá trình hiệu chỉnh camera bằng phương pháp trực tiếp là đi tìm các thông số trong ma trận chiếu (projection matrix), khi ta đã biết tọa độ các điểm $A_1, A_2, \dots, A_n$ trong hệ tọa độ vật và tọa độ tương ứng $a_1, a_2, \dots, a_n$ của chúng trên mặt phẳng ảnh.

Giả sử bỏ qua các điểm distortion, có 12 thông số cần tìm trong ma trận hình chiếu. Giả sử các điểm trong tọa độ thực là $A_i = [X_i \ Y_i \ Z_i \ 1]^T$ biết trước có ảnh $a_i = [u_i \ v_i]$ biết trước ($i = 1, 2, 3, \dots$). Ta có công thức chuyển đổi

$$\begin{bmatrix} su \\ sv \\ s \end{bmatrix} = \begin{bmatrix} m_{11} & m_{12} & m_{13} & m_{14} \\ m_{21} & m_{22} & m_{23} & m_{24} \\ m_{31} & m_{32} & m_{33} & m_{34} \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} \Rightarrow \begin{cases} su = m_{11}X + m_{12}Y + m_{13}Z + m_{14} \\ sv = m_{21}X + m_{22}Y + m_{23}Z + m_{24} \\ s = m_{31}X + m_{32}Y + m_{33}Z + m_{34} \end{cases} \quad (7)$$

Từ (7) rút ra biểu thức cho u và v

$$u = \frac{m_{11}X + m_{12}Y + m_{13}Z + m_{14}}{m_{31}X + m_{32}Y + m_{33}Z + m_{34}}$$

$$v = \frac{m_{21}X + m_{22}Y + m_{23}Z + m_{24}}{m_{31}X + m_{32}Y + m_{33}Z + m_{34}}$$

Khai triển và biến đổi chuyển về ta có biểu thức

$$0 = m_{11}X + m_{12}Y + m_{13}Z + m_{14} - m_{31}uX - m_{32}uY - m_{33}uZ - m_{34}u \quad (8)$$

$$0 = m_{21}X + m_{22}Y + m_{23}Z + m_{24} - m_{31}vX - m_{32}vY - m_{33}vZ - m_{34}v \quad (9)$$

Đưa về dạng ma trận $\mathbf{Ax} = \mathbf{0}$ ta có

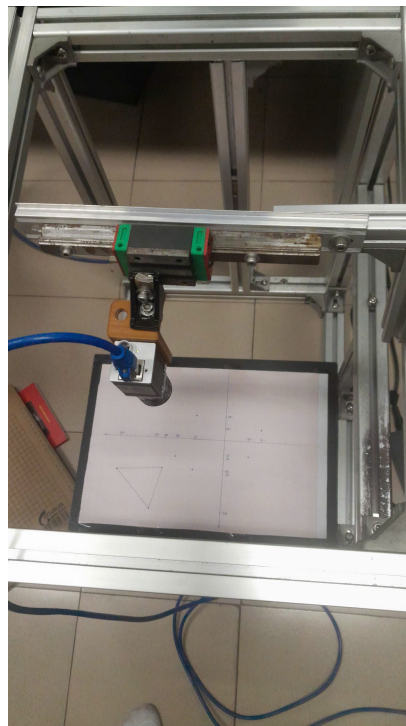
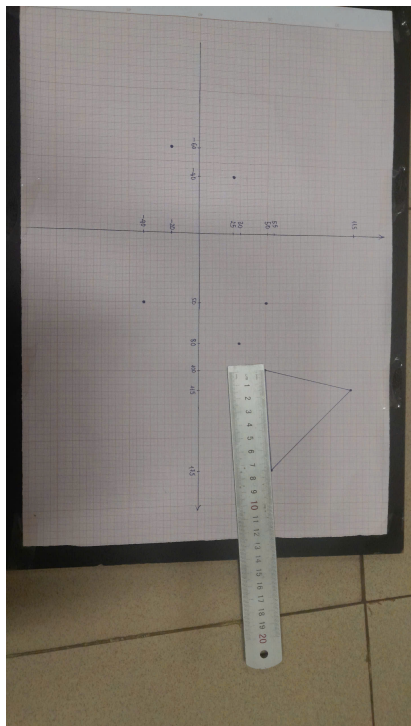
$$\begin{bmatrix} X_1 & Y_1 & Z_1 & 1 & 0 & 0 & 0 & 0 & -u_1X_1 & -u_1Y_1 & -u_1Z_1 & -u_1 \\ 0 & 0 & 0 & 0 & X_1 & Y_1 & Z_1 & 1 & -v_1X_1 & -v_1Y_1 & -v_1Z_1 & -v_1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ X_n & Y_n & Z_n & 1 & 0 & 0 & 0 & 0 & -u_nX_n & -u_nY_n & -u_nZ_n & -u_n \\ 0 & 0 & 0 & 0 & X_n & Y_n & Z_n & 1 & -v_nX_n & -v_nY_n & -v_nZ_n & -v_n \end{bmatrix} \begin{bmatrix} m_{11} \\ m_{12} \\ m_{13} \\ m_{14} \\ m_{21} \\ m_{22} \\ m_{23} \\ m_{24} \\ m_{31} \\ m_{32} \\ m_{33} \\ m_{34} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 0 \end{bmatrix} \quad (10)$$

Giải hệ này tìm ra các hệ số của ma trận hình chiếu sử dụng phương pháp SVD (Singular Value Decomposition). Rồi từ đó phân tách ra thành ma trận thông số nội K , ma trận xoay R và vector tịnh tiến T .

2.2 Các bước thực hiện

Quá trình thực hiện bao gồm các bước sau:

- Bước 1: Sử dụng một mặt phẳng chứa một hệ tọa độ xác định như hình, chiều dài mỗi ô trên hệ tọa độ là 5mm.



Hình 13: Mặt phẳng hệ tọa độ dùng để calib camera trực tiếp

- Bước 2: Chấm các điểm có tọa độ xác định trên mặt phẳng hệ tọa độ, ghi lại các tọa độ thực tế X_i, Y_i của các điểm này trong hệ tọa độ không gian.
- Bước 3: Dùng thước dây đo khoảng cách từ camera đến mặt phẳng hệ tọa độ để lấy giá trị Z_i cho các điểm đã chấm. Với mỗi độ cao, các điểm đã chấm trên mặt phẳng sẽ có tọa độ trong không gian là $A_i = [X_i \ Y_i \ Z_i \ 1]^T$.



Hình 14: Dùng thước dây đo độ cao từ camera đến mặt phẳng hệ tọa độ

- Bước 4: Chụp ảnh các điểm đã chấm từ camera, sử dụng code để lấy tọa độ ảnh u_i, v_i tương ứng của các điểm này trên mặt phẳng ảnh.

Vết hàm để thực hiện chấm các các điểm để lấy tọa độ pixel và xuất ra ma trận tọa độ trên không gian ảnh và không gian thực của từng điểm

Listing 1: Hàm đọc pixel từ điểm chấm

```
import cv2
import numpy as np

def create_calibration_data(image_path):
    """
    GUI creates calibration data by allowing the
    user to click on points in the image.
    Returns a list of pixel coordinates.
    """
    # Load image
    img = cv2.imread(image_path)
```



```
if img is None:
    print("Cannot load image.")
    return None

# List to store pixel coordinates
points = []

def mouse_callback(event, x, y, flags, param):
    if event == cv2.EVENT_LBUTTONDOWN:
        points.append((x, y))
        print(f"Clicked point: ({x}, {y})")
        cv2.circle(img, (x, y), 5, (0, 255, 0), -1)
        cv2.imshow('Image', img)

cv2.imshow('Image', img)
cv2.setMouseCallback('Image', mouse_callback)

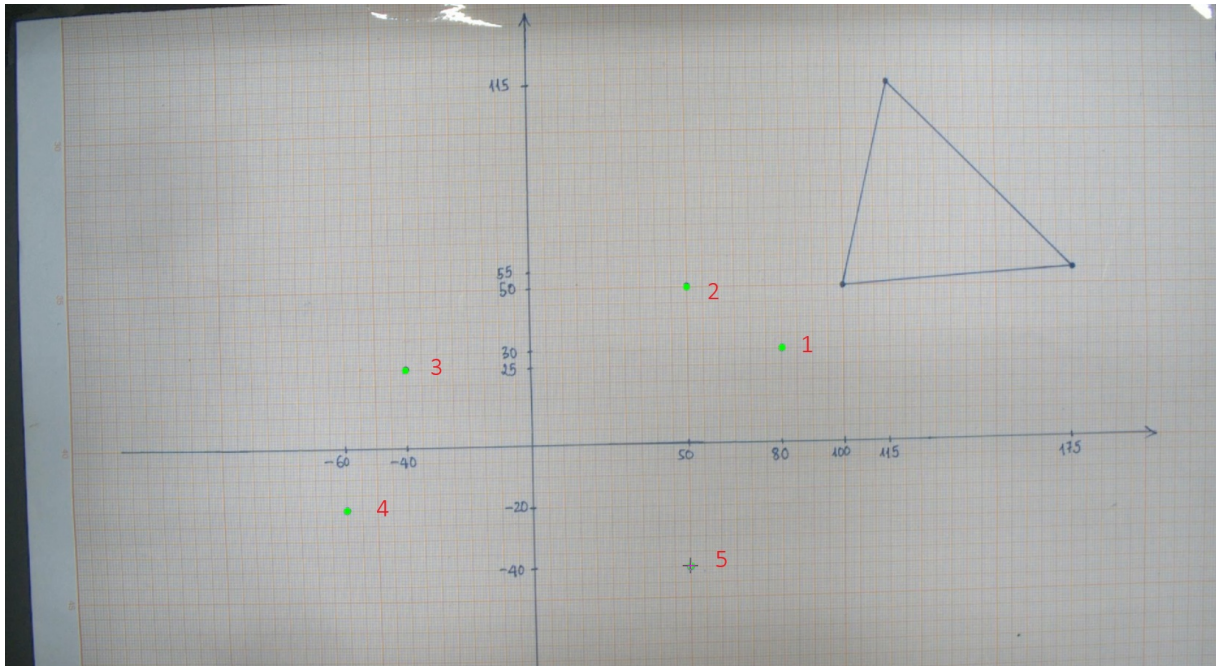
print("Click points on the image. Press 'q' to stop.")
while True:
    key = cv2.waitKey(1) & 0xFF
    if key == ord('q'):
        break

cv2.destroyAllWindows()

if not points:
    print("No points were clicked.")
    return None

return points
```

Thứ tự đánh dấu các điểm trên giao diện GUI



Hình 15: Thứ tự đánh dấu các điểm dùng để calib trên giao diện GUI

Mảng chứa dữ liệu tọa độ thực tế và tọa độ ảnh thu được sau khi chấm điểm

Bảng 1: Bảng dữ liệu tọa độ các điểm dùng để calib

u_1	v_1	x_1	y_1	z_1
1199	553	80	30	492
1057	462	50	50	492
639	587	-40	25	492
552	796	-60	-20	492
1064	880	50	-40	492
1208	552	80	30	475
1061	459	50	50	475
629	588	-40	25	475
540	804	-60	-20	475
1068	888	50	-40	475
1212	554	80	30	467
1063	458	50	50	467
624	591	-40	25	467
533	811	-60	-20	467
1070	897	50	-40	467

Sau khi có dữ liệu tọa độ thực và tọa độ ảnh, tiến hành sử dụng SVD để giải hệ phương trình và tìm ma trận chiếu. Từ ma trận chiếu ta tách ra ma trận thông số nội và ngoại.

Listing 2: Code tìm các ma trận calib

```
# Get image coordinates and world coordinates
imgCoor = data[:, :2] # columns u_1, v_1
```

```
worldCoor = data[:, 2:] # columns x_1, y_1,
                        z_1

# Build matrix P
P = np.zeros((2*len(imgCoor), 12))
j = 0

for i in range(0, 2*len(imgCoor), 2):
    P[i, 0:3] = worldCoor[j, 0:3]
    P[i+1, 4:7] = worldCoor[j, 0:3]
    P[i, 3] = 1
    P[i+1, 7] = 1
    P[i, 8:12] = -np.concatenate([worldCoor[j,
        0:3], [1]]) * imgCoor[j, 0]
    P[i+1, 8:12] = -np.concatenate([worldCoor[j,
        0:3], [1]]) * imgCoor[j, 1]
    j += 1

# SVD decomposition
U, S, V = np.linalg.svd(P)
V_last = V[-1, :]

# Create matrix M
M = np.zeros((3, 4))
M[0, 0:4] = V_last[0:4]
M[1, 0:4] = V_last[4:8]
M[2, 0:4] = V_last[8:12]

# Extract intrinsic and extrinsic matrices
# from matrix M
M = M / M[2, 3]
H = M[:, 0:3] # first 3 columns of M
T = M[:, 3]   # translation matrix

# QR decomposition to find K and Ro
Q, R = np.linalg.qr(np.linalg.inv(H))
K = np.linalg.inv(R)
Ro = Q.T

# K must have positive diagonal elements
t = np.array([[ -1, 0, 0],
              [ 0, 1, 0],
              [ 0, 0, 1]])
K = K @ t / K[2, 2]
```

Kết quả ma trận hình chiếu, ma trận thông số nội và ngoại thu được

```

Projected matrix
[[-1.66835382e-02  7.66088233e-04 -5.78996968e-03 -1.30681060e-01]
 [ 4.07786417e-04  1.69963840e-02 -3.09870761e-03 -9.91116057e-01]
 [ 1.18729419e-07  4.63444235e-07 -6.15664618e-06 -5.70505798e-04]]

Intrinsic matrix K:
[[2.71987632e+03  6.56767583e-01  8.92163113e+02]
 [0.00000000e+00  2.70743524e+03  7.08122069e+02]
 [0.00000000e+00  0.00000000e+00  1.00000000e+00]]

Extrinsics Matrix:
[[-9.99627654e-01  2.07538248e-02 -1.77153221e-02  2.29061756e+02]
 [-1.93619277e-02 -9.96963839e-01 -7.54202793e-02  1.73725852e+03]
 [-1.92267948e-02 -7.50491940e-02  9.96994458e-01  1.00000000e+00]]
    
```

Hình 16: Kết quả ma trận calib thu được từ phương pháp trực tiếp

Thực hiện kiểm tra lại kết quả ma trận calib thu được bằng cách tính toán lại tọa độ thực tế của các điểm checkpoint từ tọa độ pixel. Công thức liên hệ giữa tọa độ thực M_w và tọa độ ảnh pixel m_p là

$$M_w = R^{-1}(K^{-1}zm_p - T) \quad (11)$$

Listing 3: Kiểm tra kết quả các ma trận calib

```

# Calculate world coordinates from image
  coordinates
z_known = 492 # distance from camera to
  surface

image_path_test = r"D:\Fundamental\4\251\
  Computer_Vision\BTL\pics\b5_new\492.jpg"

worldCoor_3D = []
# imgCoor_test = np.array([[845, 507], [1462,
  100], [1750, 412]])
imgCoor_test = create_calibration_data(
  image_path_test)
imgCoor_test = np.array(imgCoor_test)
point_names = ['P1', 'P2', 'P3']

for i in range(len(imgCoor_test)):
  u = imgCoor_test[i, 0]
  v = imgCoor_test[i, 1]

# Define the system of equations
A = np.array([
  [M[0,0] - u*M[2,0], M[0,1] - u*M[2,1]],
    
```

```

[M[1,0] - v*M[2,0], M[1,1] - v*M[2,1]]
])

B = np.array([
    u*M[2,2]*z_known + u*M[2,3] - M[0,2]*
    z_known - M[0,3],
    v*M[2,2]*z_known + v*M[2,3] - M[1,2]*
    z_known - M[1,3]
])

# Solve for world coordinates (X, Y)
world_pt = np.linalg.solve(A, B)
worldCoor_3D.append([np.ceil(world_pt[0]), np.
    ceil(world_pt[1]), z_known])

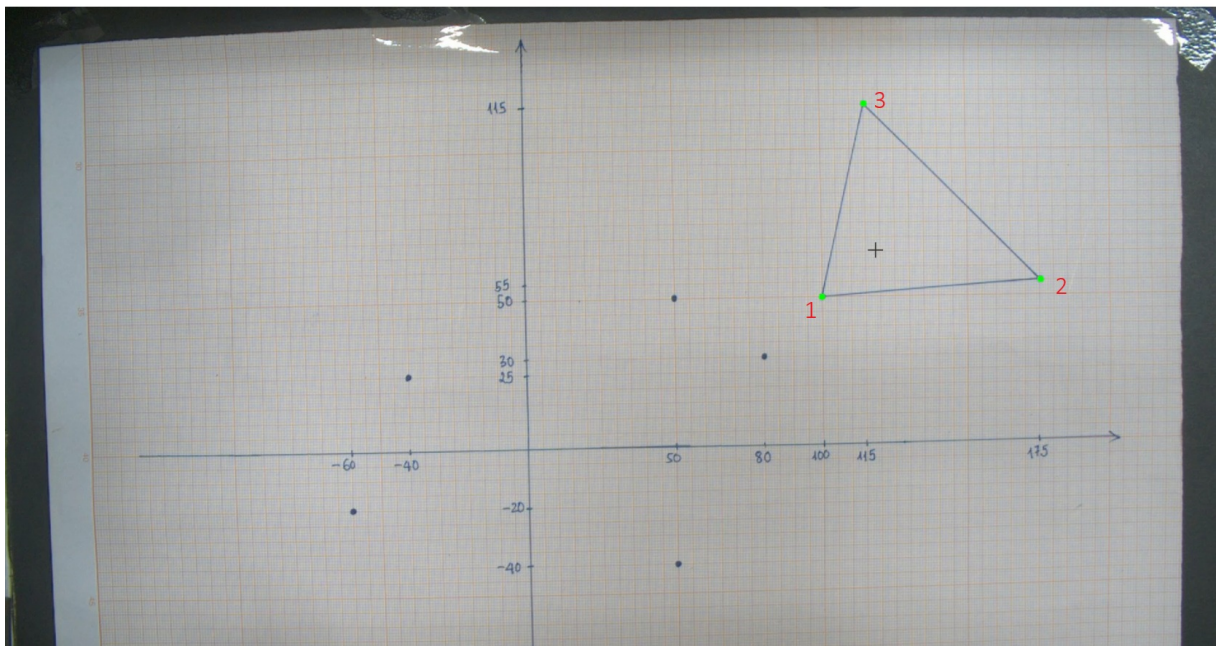
worldCoor_3D = np.array(worldCoor_3D)

# Calculate Euclidean distance between P1 and
P2
p1 = worldCoor_3D[0]
p2 = worldCoor_3D[1]

distance = np.sqrt((p2[0] - p1[0])**2 + (p2[1]
    - p1[1])**2 + (p2[2] - p1[2])**2)

```

Các điểm dùng để kiểm tra lại kết quả sau khi tính toán ma trận calib ở độ cao $h = 492mm$



Hình 17: Các điểm dùng để kiểm tra lại, độ cao $h = 492mm$

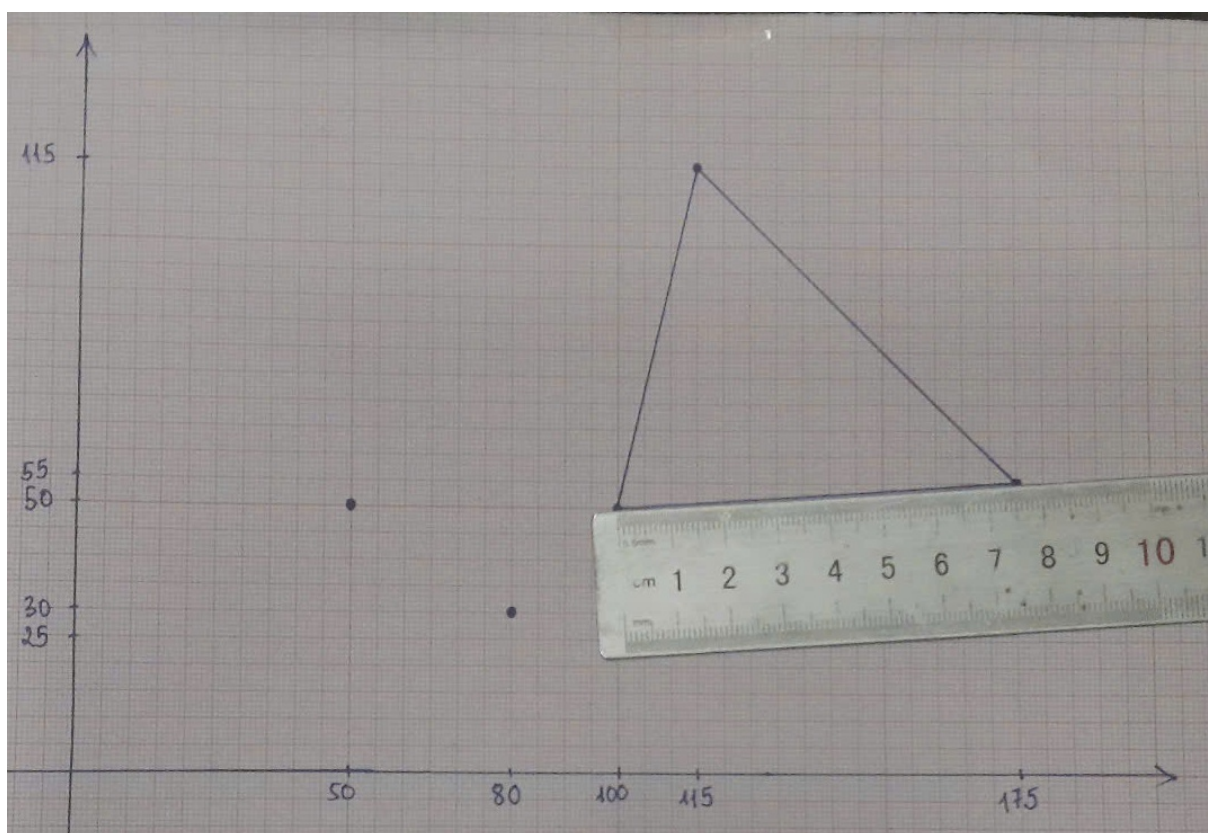
Kết quả


```
-----
Img_coordinate at P1: (1289, 459) -----> World_Coordinate: (100, 50)
Img_coordinate at P2: (1633, 431) -----> World_Coordinate: (172, 54)
Img_coordinate at P3: (1354, 155) -----> World_Coordinate: (114, 114)
-----
Distance between P1(100, 50) and P2(172, 54) with world_Coordinate: 72.111026 mm
```

Hình 18: Kết quả tọa độ các điểm trong không gian thực

So với tọa độ 3 điểm P_1, P_2, P_3 lần lượt là $(100, 50, 492)$, $(175, 55, 492)$ và $(115, 115, 492)$. Kết quả tọa độ tính toán lại từ các ma trận calib theo phương pháp trực tiếp là khá chính xác.

Khoảng cách tính toán giữa hai điểm P_1 và P_2 là 72.111026 mm, so với khoảng cách thực tế là 75 mm, kết quả thu được là khá chính xác.



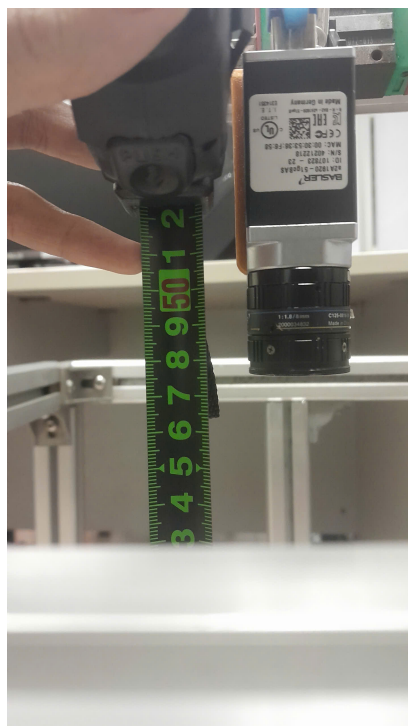
Hình 19: Kết quả khoảng cách giữa hai điểm P_1 và P_2

So sánh và nhận xét: So với ma trận thông số nội K thu được từ phương pháp calib gián tiếp, có sự sai khác giữa các hệ số f_x, f_y, c_x, c_y . Nguyên nhân có thể đến từ cách lấy các điểm dữ liệu khác nhau và nhiều trong quá trình lấy mẫu. Calib gián tiếp sử dụng checkerboard với nhiều điểm ảnh, nhiều góc độ hơn, và có tính đến distortion, ngược lại phương pháp calib camera trực tiếp lại giả định bỏ qua các yếu tố distortion và số điểm ảnh mà nhóm lấy mẫu để tính toán cũng ít hơn.

Hình ảnh độ cao từ camera đến mặt phẳng ở 3 độ cao



Hình 20: Độ cao $h_1 = 467$ mm



Hình 21: Độ cao $h_2 = 475$ mm



Hình 22: Độ cao $h_3 = 492$ mm